

SKILLWEEK8

TASK1: Parse Combined Redirection, Duplicate File Descriptors, Merge Streams, Verify Ordering, Handle Edge Cases, Test Integrated Outputs

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

#define MAX_ARGS 100
#define MAX_LINE 1024

void execute_command(char *line)
{
    char *args[MAX_ARGS];
    int argc = 0;

    char *input_file = NULL;
    char *output_file = NULL;
    char *error_file = NULL;

    int append_output = 0;
    int append_error = 0;

    char *token = strtok(line, " \\t\\n");

    while (token != NULL && argc < MAX_ARGS - 1)
    {
        if (strcmp(token, ">") == 0)
        {
            token = strtok(NULL, " \\t\\n");

            if (token == NULL)
            {
                fprintf(stderr, "Syntax error: missing output file\\n");
                return;
            }

            output_file = token;
            append_output = 0;
        }
        else if (strcmp(token, ">>") == 0)
        {
            token = strtok(NULL, " \\t\\n");

            if (token == NULL)
```

```
GNU nano 7.2
        {
            fprintf(stderr, "Syntax error: missing output file\\n");
            return;
        }

        output_file = token;
        append_output = 0;
    }
    else if (strcmp(token, ">>") == 0)
    {
        token = strtok(NULL, " \\t\\n");

        if (token == NULL)
        {
            fprintf(stderr, "Syntax error: missing output file\\n");
            return;
        }

        output_file = token;
        append_output = 1;
    }
    else if (strcmp(token, "<") == 0)
    {
        token = strtok(NULL, " \\t\\n");

        if (token == NULL)
        {
            fprintf(stderr, "Syntax error: missing input file\\n");
            return;
        }

        input_file = token;
    }
    else if (strcmp(token, "2>") == 0)
    {
        token = strtok(NULL, " \\t\\n");

        if (token == NULL)
        {
            fprintf(stderr, "Syntax error: missing error file\\n");
            return;
        }

        error_file = token;
```

```
jhansi_priya@Jhansi-Priya: ~/  + 
GNU nano 7.2
    token = strtok(NULL, " \\t\\n");
}

args[argc] = NULL;

if (argc == 0)
    return;

pid_t pid = fork();

if (pid < 0)
{
    perror("fork");
    return;
}

if (pid == 0)
{
    /*
     * Apply redirections in the order they were specified.
     * For a complete shell, redirections should ideally be
     * parsed as an ordered list. This program handles the
     * common cases.
     */

    if (input_file != NULL)
    {
        int fd = open(input_file, O_RDONLY);

        if (fd < 0)
        {
            perror("open input");
            exit(1);
        }

        dup2(fd, STDIN_FILENO);
        close(fd);
    }

    if (output_file != NULL)
    {
        int flags = O_WRONLY | O_CREAT;

        if (append_output)
            flags |= O_APPEND;
    }
}
|
```

```
jhansi_priya@Jhansi-Priya: ~/  + 
GNU nano 7.2
        new_args[j++] = args[i];
    }
    else
    {
        dup2(STDOUT_FILENO, STDERR_FILENO);
    }
}

new_args[j] = NULL;
execvp(new_args[0], new_args);
perror("execvp");
exit(1);
}
else
{
    waitpid(pid, NULL, 0);
}
}

int main()
{
    char line[MAX_LINE];

    printf("Task 1 Shell\n");
    printf("Type 'exit' to quit.\n");

    while (1)
    {
        printf("task1$ ");
        fflush(stdout);

        if (fgets(line, sizeof(line), stdin) == NULL)
            break;

        if (strncmp(line, "exit", 4) == 0)
            break;

        execute_command(line);
    }

    return 0;
}
|
```

```
jhansi_priya@Jhansi-Priya: ~/  +  
jhansi_priya@Jhansi-Priya:~$ mkdir Skill8
jhansi_priya@Jhansi-Priya:~$ cd Skill8
jhansi_priya@Jhansi-Priya:~/Skill8$ ls > output.txt
jhansi_priya@Jhansi-Priya:~/Skill8$ ls 2> error.txt
error.txt  output.txt
jhansi_priya@Jhansi-Priya:~/Skill8$ ls > output.txt 2>&1
jhansi_priya@Jhansi-Priya:~/Skill8$ ls 2>&1 > output.txt
jhansi_priya@Jhansi-Priya:~/Skill8$ cat < input.txt
-bash: input.txt: No such file or directory
jhansi_priya@Jhansi-Priya:~/Skill8$ nano task1.c
jhansi_priya@Jhansi-Priya:~/Skill8$ gcc task1.c -o task1
jhansi_priya@Jhansi-Priya:~/Skill8$ ./task1
Task 1 Shell
Type 'exit' to quit.
task1$ exit
jhansi_priya@Jhansi-Priya:~/Skill8$ ls
error.txt  output.txt  task1  task1.c
jhansi_priya@Jhansi-Priya:~/Skill8$ echo Hello > output.txt
jhansi_priya@Jhansi-Priya:~/Skill8$ cat output.txt
Hello
jhansi_priya@Jhansi-Priya:~/Skill8$ ls /wrongfolder 2> error.txt
jhansi_priya@Jhansi-Priya:~/Skill8$ cat error.txt
ls: cannot access '/wrongfolder': No such file or directory
jhansi_priya@Jhansi-Priya:~/Skill8$ ls /etc /wrongfolder > combined.txt 2>&1
1
jhansi_priya@Jhansi-Priya:~/Skill8$ cat combined.txt
ls: cannot access '/wrongfolder': No such file or directory
/etc:
PackageKit
X11
adduser.conf
alternatives
apparmor
apparmor.d
appport
apt
bash.bashrc
bash_completion
bash_completion.d
bindresvport.blacklist
```

TASK2: Parse Complex Commands, Combine Pipes and Redirections, Resolve Execution Order, Validate Syntax, Generate Execution Plans, Test Scenarios

```
GNU nano 7.2 task2.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

#define MAX_LINE 2048
#define MAX_ARGS 100
#define MAX_CMDS 20

void apply_redirections(char **args, int *argc)
{
    char *new_args[MAX_ARGS];
    int new_argc = 0;

    for (int i = 0; i < *argc; i++)
    {
        if (strcmp(args[i], "<") == 0)
        {
            if (i + 1 >= *argc)
            {
                fprintf(stderr, "Syntax error: missing input file\n");
                exit(1);
            }

            int fd = open(args[i + 1], O_RDONLY);

            if (fd < 0)
            {
                perror("open input");
                exit(1);
            }

            dup2(fd, STDIN_FILENO);
            close(fd);
```

```
dup2(fd, STDIN_FILENO);
close(fd);

        i++;
    }
    else if (strcmp(args[i], ">") == 0 ||
             strcmp(args[i], ">>") == 0)
    {
        if (i + 1 >= *argc)
        {
            fprintf(stderr, "Syntax error: missing output file\n");
            exit(1);
        }

        int flags = O_WRONLY | O_CREAT;

        if (strcmp(args[i], ">>") == 0)
            flags |= O_APPEND;
        else
            flags |= O_TRUNC;

        int fd = open(args[i + 1], flags, 0644);

        if (fd < 0)
        {
            perror("open output");
            exit(1);
        }

        dup2(fd, STDOUT_FILENO);
        close(fd);

        i++;
    }
    else if (strcmp(args[i], "2>") == 0 ||
             strcmp(args[i], "2>>") == 0)
```

```
GNU nano 7.2 task2.c *
{
    if (i + 1 >= *argc)
    {
        fprintf(stderr, "Syntax error: missing error file\n");
        exit(1);
    }

    int flags = O_WRONLY | O_CREAT;
    if (strcmp(args[i], "2>>") == 0)
        flags |= O_APPEND;
    else
        flags |= O_TRUNC;
    int fd = open(args[i + 1], flags, 0644);
    if (fd < 0)
    {
        perror("open error");
        exit(1);
    }

    dup2(fd, STDERR_FILENO);
    close(fd);

    i++;
}
else if (strcmp(args[i], "2>&1") == 0)
{
    dup2(STDOUT_FILENO, STDERR_FILENO);
}
else
{
    new_args[new_argc++] = args[i];
}
}
```

```
GNU nano 7.2 task2.c *
/*
 * Then apply redirections.
 * This means redirection can override pipe output,
 * which matches shell execution rules.
 */
apply_redirections(args, &argc);

for (int j = 0; j < command_count - 1; j++)
{
    close(pipes[j][0]);
    close(pipes[j][1]);
}

execvp(args[0], args);

perror("execvp");
exit(1);
}

for (int i = 0; i < command_count - 1; i++)
{
    close(pipes[i][0]);
    close(pipes[i][1]);
}

for (int i = 0; i < command_count; i++)
{
    waitpid(pids[i], NULL, 0);
}

return 0;
}
```

```
jhansi_priya@Jhansi-Priya:~/Skill8$ nano task2.c
jhansi_priya@Jhansi-Priya:~/Skill8$ gcc task2.c -o task2
jhansi_priya@Jhansi-Priya:~/Skill8$ ./task2
Task 2 Shell
Supports pipes and redirection.
Type 'exit' to quit.
task2$ exit
jhansi_priya@Jhansi-Priya:~/Skill8$ ls | grep txt
combined.txt
error.txt
output.txt
jhansi_priya@Jhansi-Priya:~/Skill8$ echo -e "apple\nbanana\napple\norange"
> input.txt
jhansi_priya@Jhansi-Priya:~/Skill8$ ./task2
Task 2 Shell
Supports pipes and redirection.
Type 'exit' to quit.
task2$ exit
jhansi_priya@Jhansi-Priya:~/Skill8$ cat input.txt | grep apple
apple
apple
jhansi_priya@Jhansi-Priya:~/Skill8$ cat input.txt | grep apple | wc -l
2
jhansi_priya@Jhansi-Priya:~/Skill8$ |
```

TASK3: To Detect Background Jobs, Launch Non-Blocking Processes, Return Prompt Immediately, Store Job Information, Monitor Execution, Test Background Tasks

```
GNU nano 7.2 task3.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_LINE 1024
#define MAX_ARGS 100
#define MAX_JOBS 50

typedef struct
{
    int job_id;
    pid_t pid;
    char command[MAX_LINE];
    int running;
} Job;

Job jobs[MAX_JOBS];
int job_count = 0;
int next_job_id = 1;

void check_jobs()
{
    for (int i = 0; i < job_count; i++)
    {
        if (!jobs[i].running)
            continue;

        int status;

        pid_t result = waitpid(
            jobs[i].pid,
            &status,
            WNOHANG
        );

        if (result == jobs[i].pid)
        {
            jobs[i].running = 0;
            printf("\n[%d] Done    %s\n",
                jobs[i].job_id,
                jobs[i].command);
        }
    }
}
```

```
GNU nano 7.2 task3.c *
        jobs[i].job_id,
        jobs[i].command);
        fflush(stdout);
    }
}

void show_jobs()
{
    check_jobs();

    for (int i = 0; i < job_count; i++)
    {
        printf("[%d] PID=%d  %s  %s\n",
            jobs[i].job_id,
            jobs[i].pid,
            jobs[i].running ? "Running" : "Done",
            jobs[i].command);
    }
}

int main()
{
    char line[MAX_LINE];

    printf("Task 3 Background Job Shell\n");
    printf("Commands ending with & run in background.\n");
    printf("Type 'jobs' to view jobs.\n");
    printf("Type 'exit' to quit.\n");

    while (1)
    {
        check_jobs();

        printf("task3$ ");
        fflush(stdout);

        if (fgets(line, sizeof(line), stdin) == NULL)
            break;

        line[strcspn(line, "\n")] = '\0';

        if (strcmp(line, "exit") == 0)
```

```
GNU nano 7.2 task3.c *
break;
line[strcspn(line, "\n")] = '\0';
if (strcmp(line, "exit") == 0)
    break;
if (strcmp(line, "jobs") == 0)
{
    show_jobs();
    continue;
}
if (strlen(line) == 0)
    continue;
int background = 0;
int len = strlen(line);
/*
 * Check whether command ends with &
 */
if (line[len - 1] == '&')
{
    background = 1;
    line[len - 1] = '\0';
    while (len > 0 && line[len - 1] == ' ')
    {
        line[len - 1] = '\0';
        len--;
    }
}
char original_command[MAX_LINE];
strcpy(original_command, line);
char *args[MAX_ARGS];
int argc = 0;
char *token = strtok(line, " \t");
while (token != NULL && argc < MAX_ARGS - 1)
```

```
GNU nano 7.2 task3.c *
if (background)
{
    if (job_count < MAX_JOBS)
    {
        jobs[job_count].job_id = next_job_id++;
        jobs[job_count].pid = pid;
        jobs[job_count].running = 1;

        strcpy(
            jobs[job_count].command,
            original_command
        );
        printf("[%d] %d\n",
            jobs[job_count].job_id,
            pid);

        job_count++;
    }
}
else
{
    /*
     * Foreground command.
     * Shell waits for it.
     */
    waitpid(pid, NULL, 0);
}

/*
 * Wait for remaining background processes before exiting.
 */
for (int i = 0; i < job_count; i++)
{
    if (jobs[i].running)
    {
        waitpid(jobs[i].pid, NULL, 0);
    }
}

return 0;
}
```

```
jhansi_priya@Jhansi-Priya:~/Skill8$ nano task3.c
jhansi_priya@Jhansi-Priya:~/Skill8$ gcc task3.c -o task3
jhansi_priya@Jhansi-Priya:~/Skill8$ ./task3
Task 3 Background Job Shell
Commands ending with & run in background.
Type 'jobs' to view jobs.
Type 'exit' to quit.
task3$ exit
jhansi_priya@Jhansi-Priya:~/Skill8$ echo Hello
Hello
jhansi_priya@Jhansi-Priya:~/Skill8$ sleep 10 &
[1] 910
jhansi_priya@Jhansi-Priya:~/Skill8$ jobs
[1]+  Running                  sleep 10 &
jhansi_priya@Jhansi-Priya:~/Skill8$ jobs
[1]+  Done                      sleep 10
jhansi_priya@Jhansi-Priya:~/Skill8$
```

TASK4: To Create Job Table, Store Process Groups, Track States, Update Status, Remove Completed Jobs, Validate Job Records

```
GNU nano 7.2 task4.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <signal.h>

#define MAX_LINE 1024
#define MAX_ARGS 100
#define MAX_JOBS 50

typedef enum
{
    RUNNING,
    STOPPED,
    DONE
} JobState;

typedef struct
{
    int job_id;
    pid_t pid;
    pid_t pgid;
    JobState state;
    char command[MAX_LINE];
} Job;

Job jobs[MAX_JOBS];
int job_count = 0;
int next_job_id = 1;

const char *state_name(JobState state)
{
    switch (state)
    {
        case RUNNING:
            return "Running";

        case STOPPED:
            return "Stopped";

        case DONE:
            return "Done";

        }
    return "Unknown";
}

void update_jobs()
{
    for (int i = 0; i < job_count; i++)
    {
        if (jobs[i].state == DONE)
```

```
        continue;

        int status;

        pid_t result = waitpid(
            jobs[i].pid,
            &status,
            WNOHANG | WUNTRACED | WCONTINUED
        );

        if (result == 0)
        {
            continue;
        }

        if (result == jobs[i].pid)
        {
            if (WIFEXITED(status) ||
                WIFSIGNALED(status))
            {
                jobs[i].state = DONE;
                printf("\n[%d] Done %s\n",
                    jobs[i].job_id,
                    jobs[i].command);

                fflush(stdout);
            }
            else if (WIFSTOPPED(status))
            {
                jobs[i].state = STOPPED;
                printf("\n[%d] Stopped %s\n",
                    jobs[i].job_id,
                    jobs[i].command);

                fflush(stdout);
            }
            else if (WIFCONTINUED(status))
            {
                jobs[i].state = RUNNING;
            }
        }
    }
}

void show_jobs()
{
    update_jobs();
    printf("\n");
    printf("JOB\tPID\tPGID\tSTATE\tCOMMAND\n");
```

```
GNU nano 7.2 task4.c *
for (int i = 0; i < job_count; i++)
{
    printf("%d\t%d\t%d\t%d\t%-10s\t%s\n",
           jobs[i].job_id,
           jobs[i].pid,
           jobs[i].pgid,
           state_name(jobs[i].state),
           jobs[i].command);
}
printf("\n");

void remove_completed_jobs()
{
    int new_count = 0;
    for (int i = 0; i < job_count; i++)
    {
        if (jobs[i].state != DONE)
        {
            jobs[new_count++] = jobs[i];
        }
    }
    job_count = new_count;
}

int main()
{
    char line[MAX_LINE];
    printf("Task 4 Job Table Shell\n");
    printf("Supports background jobs and process groups.\n");
    printf("Commands:\n");
    printf("  jobs      - show job table\n");
    printf("  clean     - remove completed jobs\n");
    printf("  exit      - exit shell\n");

    while (1)
    {
        update_jobs();

        printf("task4$ ");
        fflush(stdout);

        if (fgets(line, sizeof(line), stdin) == NULL)
            break;

        line[strcspn(line, "\n")] = '\0';

        if (strcmp(line, "exit") == 0)
            break;
    }
}
```

```
GNU nano 7.2 task4.c *
/* its own process group.
 */
setpgid(pid, pid);

if (background)
{
    if (job_count >= MAX_JOBS)
    {
        printf("Job table full.\n");
        kill(pid, SIGTERM);
        continue;
    }

    jobs[job_count].job_id = next_job_id++;
    jobs[job_count].pid = pid;
    jobs[job_count].pgid = pid;
    jobs[job_count].state = RUNNING;

    strcpy(
        jobs[job_count].command,
        original_command
    );

    printf("[%d] PID=%d PGID=%d\n",
           jobs[job_count].job_id,
           pid,
           pid);

    job_count++;
}
else
{
    /* Foreground process.
    */
    waitpid(pid, NULL, 0);
}
}

/* Wait for remaining jobs.
 */
for (int i = 0; i < job_count; i++)
{
    if (jobs[i].state != DONE)
    {
        waitpid(jobs[i].pid, NULL, 0);
    }
}

return 0;
}
```

```
jhansi_priya@Jhansi-Priya: ~ × + ∨  
jhansi_priya@Jhansi-Priya:~$ ps -o pid,ppid,pgid,stat,cmd  
PID    PPID    PGID  STAT  CMD  
964     957     964   Ss    -bash  
977     964     977   R+    ps -o pid,ppid,pgid,stat,cmd
```

```
jhansi_priya@Jhansi-Priya: ~/ × + ∨  
jhansi_priya@Jhansi-Priya:~/Skill8$ gcc task4.c -o task4  
jhansi_priya@Jhansi-Priya:~/Skill8$ ./task4  
Task 4 Job Table Shell  
Supports background jobs and process groups.  
Commands:  
  jobs    - show job table  
  clean   - remove completed jobs  
  exit    - exit shell  
  
task4$ exit  
jhansi_priya@Jhansi-Priya:~/Skill8$ sleep 20 &  
[1] 1017  
jhansi_priya@Jhansi-Priya:~/Skill8$ jobs  
[1]+  Running                  sleep 20 &  
jhansi_priya@Jhansi-Priya:~/Skill8$ sleep 10 &  
sleep 20 &  
sleep 30 &  
[2] 1022  
[1]   Done                      sleep 20  
[3] 1023  
[4] 1024  
jhansi_priya@Jhansi-Priya:~/Skill8$ jobs  
[2]   Running                  sleep 10 &  
[3]-  Running                  sleep 20 &  
[4]+  Running                  sleep 30 &  
jhansi_priya@Jhansi-Priya:~/Skill8$ |
```

